

Iniciando projetos

Expo manager workflow / Projeto gerenciado (manager) / Fluxo gerenciado

Criando o Projeto

1 - Acesse a pasta que deseja iniciar o projeto pelo PoweShell

Logo após execute o seguinte código:

```
"npx create-expo-app --template
```

Caso de algum erro pode ser a segurança do Windows, execute o comando e teste novamente:

```
"Set-ExecutionPolicy RemoteSigned -Scope CurrentUser"
```

2 – Logo após a execução vai aparecer as seguintes opções:

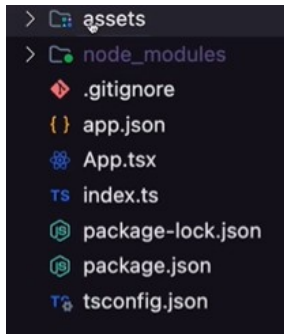
```
PS C:\Users\User\Desktop\CURSO REACT NATIVE\projeto REACT> npx create-expo-app --template
? Choose a template: » - Use arrow-keys. Return to submit.
> Default - includes tools recommended for most app developers
  Blank
  Blank (TypeScript)
  Navigation (TypeScript)
  Blank (Bare)
```

Podemos escolher a opção "Blank (TypeScript)" para iniciar o projeto em branco que já vem com uma estrutura mínima de um projeto.

3 – Após isso aparece a opção de dar o nome do projeto. "Só digitar e clicar no OK"

Será criado um diretório com o nome escolhido para o projeto, dentro dessa pasta será criada a estrutura do projeto, após a criação podemos abrir a IDE no diretório.

4 – IDE e estrutura de dados.



Assets – Ícone, Splash-Icon imagens quando o app está carregando, fivecon imagem para WEB

Node_Modules – Os módulos disponíveis para a aplicação, conforme adicionarmos novos módulos ficarão armazenados nesta pasta também. (Esta pasta pode ser deletada, ela não precisa subir no Git hub)

.gitignore – Lista de arquivos que o Git pode ignorar ao subir, inclusive o “node_modules” já está na lista.

App.json – Informações / Configuração da a aplicação, e configurações específicas do Android E IOS.

App.tsx – Existe uma função “app” que retorna o conteúdo que será renderizado na tela. “é um componente”

Index.ts – Ponto de entrada da aplicação. Onde registramos como Root nosso componente “app.tsx”, onde não precisamos colocar ele como extensão e só o nome “./app”;

Package-lock.json – Contém as dependência de todas as versões que foi utilizada para a criação

Package.json – Contém o Scripts para iniciar a aplicação e tem as dependências “módulos, dependências de produção e desenvolvimento”.

Tsconfig.json – Configurações do Type Script, para trabalhar com a Tipagem.

Modulos:

Como temos todas as dependências do APP no “Package-lock.json e Package.json” podemos apagar o “node_modules” e reinstalar facilmente pelo terminal utilizando o seguinte código:

```
npm i
```

Ele vai gerar a pasta “node_modules” novamente só com as dependências do APP

Executando a aplicação no Celular

No terminal no diretório da aplicação execute o seguinte código:

```
npx expo start
```



Vai apare diversas opções para escolher aonde e como deseja rodar o aplicativo, para funcionar no Expo Go é só scanear o QR-Code

Execuando na Web:

```
npx expo install react-dom react-native-web @expo/metro-runtime
```

```
npx expo start
```

Executando ele na “web” também podemos acessar pelo celular em quanto o aplicativo estiver rodando.

Estilização

```
<View style={{ flex:1, justifyContent:"center"}}>
```

Princípio da estilização:

Estilização é formada por objetos e dentro desses objetos recebem os seguintes valores
{propriedade: valor} = {flex: 1}, {JustifyContent: "center"}

Valor: Pode ser um número ou um texto, se for um texto tem que estar entre aspas como no exemplo a cima.

Estilização em Line:

Conseguimos estilizar através de propriedade **"style"**, dentro dessa linha colocamos todas as estilizações dentro de objetos, a primeira "chave {}" é aonde deve ficar os objetos de estilização, então dentro da primeira chave podemos ter várias outras cada um será um objeto.

Não é a mais recomendada, pois se for colocar bastante propriedades a linha vai ficar enorme.

```
<View style={{ flex:1, justifyContent:"center"}}>
```

Estilização em Const (Constante):

- 1 - Importa o StyleSheet do React Native para criar a estilização.
- 2 - Declara o "const" dentro de um arquivo (**styles.ts**) ou declara fora da função "App()".
- 3 - Dentro da const criamos os objetos de estilização, cada objeto é separado por virgula.
- 4 - Declara o objeto criado dentro da "tag" que deseja estilizar.

Declaração do "Const"

```
const style = StyleSheet.create({  
  container: {  
    flex:1,  
    justifyContent:"center",  
    alignItems: "center",  
  }  
})
```

Declaração do objeto na "tag"

```
<View style={style.container}>
```

Declaração do Const:

Quando criamos o arquivo (Styles.ts) precisamos realizar as devidas importações.

- **Arquivo Styles.ts** é importante antes do const declarar o “**export**” (chamado de exportação nomeada), que indica que o const será exportado.

```
export const styles = StyleSheet.create({
```

- **Arquivo App.tsx** importa o arquivo Style.ts

```
import {styles} from "./styles"
```

BIZU MASSTEEEEEEEEEEEEER

Organização de arquivos

PASTA “src”

Criar pasta “src” e dentro dessa pasta colocamos todos os arquivos criados por nós.

Dentro da pasta src podemos ir organizando nossos arquivos em partes, exemplo podemos criar a pasta “app” e dois “home” para colocar nossas telas home do app e mover para lá a estilização “styles.ts” e o “App.tsx”

Quando realizamos a mudança de arquivos para alguma pasta é sempre importante se atentar aos códigos de importação, provavelmente os códigos começarão a dar erro por não localizar os arquivos.

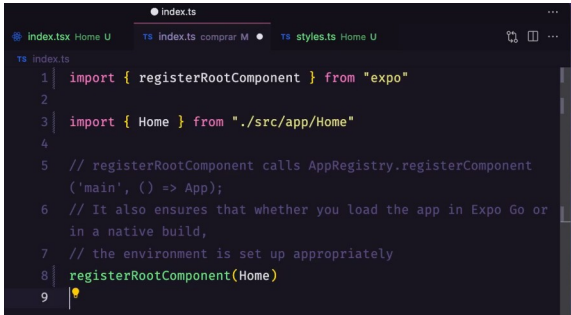
Ex: index.ts não conseguirá importar o “`import App from './App';`”, porque não estará mais na raiz do projeto, será necessário passar o caminho correto da pasta:

```
import App from './src/app/home/App';
```

Padrões de arquivos:

Index: responsável pela estrutura.

Quando temos um “**index.tsx**” dentro da pasta (no caso a Home), não precisamos declarar ele dentro do arquivo “**index.ts**”, só precisamos declarar o caminho até a “Home” e realizar a importação nomeada, como na imagem:

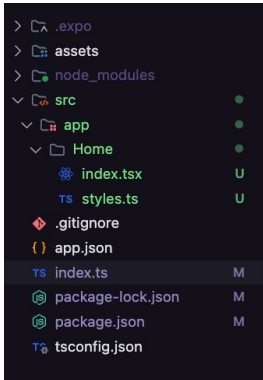
A screenshot of a code editor showing the content of index.ts. The code imports registerRootComponent from 'expo' and Home from './src/app/Home'. It then calls registerRootComponent(Home) at the bottom.

```
1 | import { registerRootComponent } from "expo"
2 |
3 | import { Home } from "./src/app/Home"
4 |
5 | // registerRootComponent calls AppRegistry.registerComponent
6 | // ('main', () => App);
7 | // It also ensures that whether you load the app in Expo Go or
8 | // in a native build,
9 | // the environment is set up appropriately
10 | registerRootComponent(Home)
```

“intex.ts”

Styles: responsável pela estilização.

Assim ficou a estrutura de arquivos:

A screenshot of a file explorer showing the project structure. The 'src' folder is expanded, showing 'app' and 'Home' subfolders. 'app' contains 'index.tsx' and 'styles.ts'. 'Home' contains 'index.tsx' and 'styles.ts'. Other files like .gitignore, app.json, index.ts, package-lock.json, package.json, and tsconfig.json are also visible.

```
> .expo
> assets
> node_modules
  src
    app
      index.tsx
      styles.ts
    Home
      index.tsx
      styles.ts
  .gitignore
  app.json
  index.ts
  package-lock.json
  package.json
  tsconfig.json
```

Dentro da pasta “Home” ficou os arquivos “index.tsx e style.ts”.

Bizu

React Native utiliza Camewcase bem parecido com o CSS;

Começa escrevendo tudo minúsculo e se for um nome composto o segundo nome começa com letra maiúscula

Ex: justfyContent